COMS W4701: Artificial Intelligence, Summer 2026

Homework 3

Instructions: Prepare submissions **only** for problems marked "Graded". Compile written responses in a single, typed PDF file. Coding solutions may be directly implemented in the provided Python file(s). Do not modify any filenames or code outside of the indicated sections. Submit all rquired files on Pensive in the appropriate assignment bins. Please be mindful of the deadline, as well as our policies on citations and academic honesty.

[No Submission] Problem 1: Reinforcement Learning

The performance of the robot from HW2 has degraded, and the original model no longer appears to be valid. In particular, the *off* state is now a terminal state. Suppose we observe the following two episodes of state and reward sequences following the policy π of going *slow* in both states.

- Episode 1: $high, 2, high, 1, low, 1, low, 1, low, 0, off$
- Episode 2: $high, 2, high, 2, high, 1, low, 0, off$

1. Suppose we treat each episode as a single sample for the *high* state. Using $\gamma = 0.5$, compute the sample values $G_1(high)$ and $G_2(high)$. Use them to compute the estimated state value $V^\pi(high)$.
2. Now consider running TD(0) starting with initial values of 0 for all three states, using $\gamma = 0.5$ and $\alpha = 0.8$. Show the updates that occur over the course of Episode 1. There should be a total of five value updates.
3. We want the robot to try different actions to learn a better policy. It has the following Q-values so far: $Q(high, fast) = 2, Q(high, slow) = 0, Q(low, fast) = -1, Q(low, slow) = 1$. What is its current greedy policy? If we know that all possible future rewards are nonnegative, is it possible for the robot to learn a different policy if it always acts greedily and never explores?
4. We send the robot off to do some active reinforcement learning. It has the following Q-values so far: $Q(high, fast) = 2, Q(high, slow) = 0, Q(low, fast) = -1, Q(low, slow) = 1$. Starting off in the *high* state, it takes the greedy action, receives a reward of +2, and lands in the *low* state. It then takes the exploratory action, receives a reward of +1, and stays in the *low* state. Show the resultant Q-value updates performed by the Q-learning algorithm, using $\gamma = 0.5$ and $\alpha = 0.8$.

[No Submission] Problem 2: Recommendation Model

A certain app is deciding whether to show you a personalized **ad**, represented by a binary random variable A. It can possibly consider two factors: whether you browse the **virtual** marketplace and whether you make a trip to the **physical** store location (it can track your location!). These

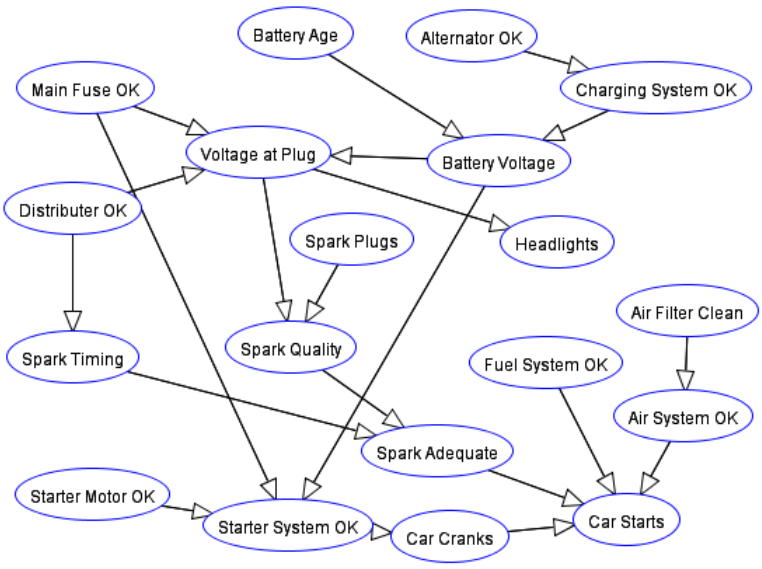
decisions are represented by binary random variables V and P. The joint distribution over all three variables is as follows:

V	P	A	$\Pr(V, P, A)$
$+v$	$+p$	$+a$	0.12
$+v$	$+p$	$-a$	0.08
$+v$	$-p$	$+a$	0.18
$+v$	$-p$	$-a$	0.12
$-v$	$+p$	$+a$	0.06
$-v$	$+p$	$-a$	0.14
$-v$	$-p$	$+a$	0.09
$-v$	$-p$	$-a$	0.21

1. Find the marginal distributions of each of the three random variables.
2. Find the joint distributions of each of the three different *pairs* of the random variables.
3. Using the distributions you found above, show whether each pair of random variables is independent.
4. Find the following conditional distributions: $\Pr(V|+a)$, $\Pr(P|+a)$, $\Pr(V, P|+a)$.
5. Can we conclude that V and P are conditionally independent given A without further calculations? If not, what additional calculations do we need to verify?

[No Submission] Problem 3: Car Start Bayes Net

This problem investigates the "Car Starting" Bayes net from the Sample Problems of the Belief and Decision Networks tool on AIspace.



1. Suppose we observe no variables. What variables, if any, are guaranteed to be independent of "Starter System OK"?

2. (i) Suppose we observe "Battery Voltage" and "Spark Adequate". (i) Relative to (a), which variables, if any, gain guarantees of independence to "Starter System OK"? (ii) Which variables, if any, lose guarantees of independence to "Starter System OK"?

3. Write an analytical expression for the marginal distribution Pr(Voltage at Plug). Your expression should only include Bayes net parameters (CPTs) and sum and product operations. You may abbreviate the variable names using just the first letters of the words, i.e. Pr(VAP) in place of Pr(Voltage at Plug).

4. Repeat the above for Pr(Spark Adequate|voltage at plug). "voltage at plug" represents an *observed* variable.

[Graded] Problem 4: Crawler Robot

You will be training a simple crawler robot modeled as a MDP. When you download the accompanying code files and run `python crawler.py` in your terminal, you should see a GUI pop with a robot on the left side of the screen. It consists of a rigid rectangular body and two joints (an "arm" and a "hand") that can be moved in discrete increments. The state space consists of discrete joint angle combinations, and the action set in each state allows the robot to move one of the joints either up or down. You should also see some buttons on the top third of the GUI that will allow you to adjust various parameters. The robot is initially stuck, but it will eventually learn how to move forward by moving its joints and pushing off the ground below it.

Part 1: Dynamic Programming (20 points)

One method for learning a good policy is to do so entirely offline using dynamic programming. In `DP_Agent.py`, we define a `DP_Agent` class. A `DP_Agent` stores the set of states, a set of values, and a policy. It is also initialized with a discount factor `gamma`. To perform dynamic programming, you will implement policy iteration, a variant of value iteration.

First write `policy_evaluation()`. This will iteratively compute the values for the current policy stored in `self.policy` and store them in `self.values`. The `transition` argument is a `Callable` that returns the successor state and reward given a state and action (all transitions are deterministic). If `None` is returned as a successor state, you can use 0 as its "value". Convergence occurs when the maximum change in any state value is no greater than 10^{-6}.

Next, write `policy_extraction()`, which will store the corresponding actions for all states in the `self.policy` dictionary, computed using the current `state.values`. In addition to `transition`, it also takes in a `valid_actions` argument, a `Callable` that returns a list of all actions for a given state. Note that this is the same procedure that would be run after completion of a value iteration routine, although here we will be using this even when the values are not yet optimal.

Finally, write `policy_iteration()`. This will repeatedly call the two subroutines that you wrote above: `policy_evaluation()`, then `policy_extraction()`. This stops when the "old" policy in `self.policy` *does not change* after running both subroutines. This signals convergence to an optimal policy, equivalent to value convergence in value iteration.

Once you finish all methods, you can test your implementation by running `python crawler.py`. If all goes well, your robot should be able to start moving across the screen with its newfound policy.

Part 2: Reinforcement Learning (18 points)

A second approach for learning a policy is to do so online using reinforcement learning. In `RL_Agent.py`, we define a `DP_Agent` class. A `DP_Agent` stores the set of states, a set of Q-values, and the parameters `alpha`, `epsilon`, and `gamma`.

First write the `choose_action()` method, which performs ε-greedy action selection given the `state` and `valid_actions` list. It should make reference to `Qvalues` if deciding to behave greedily. Next, write the `update()` method, which makes a Q-learning update to the appropriate Q-value given all components of a single transition and the `valid_actions` of the `successor` state. As in Part 1 above, if the successor is `None`, you may set its "Q-value" to 0.

You can test your implementation by running `python crawler.py -q`. The robot will appear to struggle on its own for a while, but after enough time passes it should start moving more regularly. You can decrease the "Time per action" setting at the top left to make the simulation run faster.

Part 3: Analysis (12 points)

1. Let the `DP_agent` run for at least 200 steps. What is the robot's 100-step average velocity? How does this change when you a) increase `gamma` to at least 0.9, and b) decrease it below 0.7 (give it time to settle after each change)? Describe how `gamma` affects the robot's performance.

2. Let the `RL_agent` train until it crosses the screen at least once so that it has learned an optimal or near-optimal policy. Describe how its performance changes when you a) increase and b) decrease `epsilon` by at least 0.25 in each direction.

3. Let the `RL_agent` train until it crosses the screen at least once, and note approximately how many steps it took to do so. Then start a new run and decrease `alpha` to about 0.1 at the very beginning. Describe the effect of the learning rate on the robot's training time.

Submission

Please submit the `DP_Agent.py` and `RL_Agent.py` files on Pensive.